드론 탐지 및 추적 시스템 완벽 가이드

jetson_sparse_dasiamrpn_v3.py 상세 매뉴얼

📚 목차

1. 시스템 개요
2. 핵심 개념 이해
3. 코드 구조 전체 흐름
4. 클래스별 상세 설명
5. 주요 메소드 라인별 분석
6. 실행 흐름도
7. 파라미터 조정 가이드
8. 문제 해결 가이드

시스템 개요

🎯 이 코드가 하는 일

이 시스템은 **드론을 탐지하고 추적**하는 프로그램입니다. 두 가지 핵심 기술을 조합합니다:

1. **YOLO (객체 탐지)**
   - 매 프레임마다 이미지에서 드론을 찾습니다
   - 빠르고 정확하지만, 가끔 놓칠 수 있습니다
   - 역할: "드론이 어디 있나요?"
2. **DaSiamRPN (객체 추적)**
   - 한 번 찾은 드론을 계속 따라갑니다
   - YOLO가 놓쳤을 때 백업으로 사용
   - 역할: "마지막에 본 드론이 지금 어디로 갔나요?"

💡 핵심 아이디어: "Sparse (희소) 추적"

문제: DaSiamRPN을 매 프레임마다 실행하면 너무 느림 (18 FPS)

해결:

- YOLO는 매 프레임 실행 (빠름)
- DaSiamRPN은 3프레임마다 한 번만 실행
- 나머지 프레임은 캐시된 결과 재사용

결과: 50+ FPS로 빠르면서도 정확!

핵심 개념 이해

📦 Bounding Box (바운딩 박스)

물체를 감싸는 직사각형 박스입니다.

```
형식: [x, y, w, h]
- x: 박스 왼쪽 상단 모서리의 x 좌표
- y: 박스 왼쪽 상단 모서리의 y 좌표
- w: 박스의 너비 (width)
- h: 박스의 높이 (height)

예시: [100, 50, 30, 40]
→ x=100, y=50 위치에서 시작
→ 너비 30픽셀, 높이 40픽셀인 박스
```

🎮 3가지 상태 (Status)

프로그램은 현재 어떤 방법으로 드론을 찾았는지 표시합니다:

1. **DETECTED** (초록색)
   - YOLO가 드론을 찾음
   - 가장 신뢰할 수 있는 상태
2. **TRACKED** (주황색)
   - YOLO가 실패하여 DaSiamRPN이 추적 중
   - 백업 상태
3. **LOST** (빨간색)
   - 둘 다 실패
   - 드론을 못 찾음

⚡ Sparse Tracking (희소 추적) 전략

```
프레임별 실행 패턴 (tracking_interval=3일 때):

Frame 1: YOLO (13ms) + DaSiamRPN (18ms) = 31ms
Frame 2: YOLO (13ms) + 캐시 (0ms)       = 13ms ← 빠름!
Frame 3: YOLO (13ms) + 캐시 (0ms)       = 13ms ← 빠름!
Frame 4: YOLO (13ms) + DaSiamRPN (18ms) = 31ms
Frame 5: YOLO (13ms) + 캐시 (0ms)       = 13ms ← 빠름!
...

평균: (31 + 13 + 13) / 3 = 19ms → 52 FPS!
```

예외: YOLO가 실패하면 매 프레임 DaSiamRPN 실행 (정확도 우선)

코드 구조 전체 흐름

```
프로그램 시작
    ↓
[1] 모델 로딩
    - YOLO 모델 로드
    - DaSiamRPN 네트워크 로드
    - GPU 초기화
    ↓
[2] 비디오/카메라 열기
    - 입력 소스 확인
    - 해상도, FPS 설정
    ↓
[3] (선택사항) 수동 초기화
    - 사용자가 첫 프레임에서 드론 선택
    - 트래커 초기화
    ↓
[4] 메인 루프 시작
    │
    ├─▶ 프레임 읽기
    │   ↓
    ├─▶ process_frame() 호출
    │   │
    │   ├─▶ YOLO 탐지 실행
    │   │   ↓
    │   ├─▶ 추적 전략 결정
    │   │   - 탐지 성공? → Sparse 추적
    │   │   - 탐지 실패? → 매 프레임 추적
    │   │   ↓
    │   ├─▶ 결과 검증
    │   │   - 탐지와 추적 비교
    │   │   - Jump 체크
    │   │   ↓
    │   └─▶ 최종 bbox 결정
    │       ↓
    ├─▶ 화면에 그리기
    │   ↓
    ├─▶ ESC 누름? → 종료
    │   ↓
    └─▶ 반복
```

클래스별 상세 설명

📌 SparseDaSiamRPNTracker 클래스

이 클래스가 모든 핵심 로직을 담당합니다.

🔧 초기화 메소드 (`__init__`)

목적: 시스템 시작 시 필요한 모든 것을 준비

```
def __init__(self, detector_path, conf_threshold=0.5, jump_threshold=100,
             dasiamrpn_model='./DaSiamRPN/code/SiamRPNVOT.model',
             tracking_interval=3):
```

파라미터 설명:

파라미터	기본값	의미	수정 시 영향
`detector_path`	필수	YOLO 모델 파일 경로	다른 모델 사용 시 변경
`conf_threshold`	0.5	탐지 신뢰도 임계값	높이면 더 확실한 것만 탐지
`jump_threshold`	100	허용 최대 이동 거리 (픽셀)	높이면 갑작스런 이동 허용
`dasiamrpn_model`	경로	DaSiamRPN 모델 파일	변경 불필요
`tracking_interval`	3	추적 실행 주기 (프레임)	낮추면 정확, 높이면 빠름

라인별 분석:

```
# 라인 37-38: YOLO 모델 로드
print(f"🔍 Loading YOLO model: {detector_path}")
self.detector = YOLO(detector_path)
# → YOLO 객체 탐지 모델을 메모리에 불러옴

# 라인 39-41: 파라미터 저장
self.conf_threshold = conf_threshold  # 탐지 임계가ㅅ
self.jump_threshold = jump_threshold   # 점프 임계가ㅅ
self.tracking_interval = tracking_interval  # 추적 주기
# → 나중에 사용할 설정가ㅅ들을 저장

# 라인 44-47: DaSiamRPN 네트워크 로드
print(f"🧠 Loading DaSiamRPN model: {dasiamrpn_model}")
self.net = SiamRPNvot()  # 네트워크 구조 생성
self.net.load_state_dict(torch.load(dasiamrpn_model))  # 학습된 가중치 로드
self.net.eval()  # 평가 모드로 전환 (학습 안 함)
# → DaSiamRPN 딥러닝 모델을 준비

# 라인 49-53: GPU 사용 여부 확인
if torch.cuda.is_available():
    self.net = self.net.cuda()  # GPU로 이동
    print("🚀 DaSiamRPN using CUDA")
else:
    print("💻 DaSiamRPN using CPU")
# → GPU가 있으면 사용 (훨씬 빠름)

# 라인 55-58: 추적 상태 초기화
self.dasiamrpn_state = None  # DaSiamRPN 내부 상태
self.tracking_initialized = False  # 추적 시작 여부
self.last_track_bbox = None  # 마지막 추적 결과
self.track_frame_count = 0  # 프레임 카운터
```

```
# → 추적 관련 변수들 초기화

# 라인 61-67: 통계 변수 초기화
self.frame_count = 0  # 전체 프레임 수
self.detection_used = 0  # 탐지 사용 횟수
self.tracking_used = 0  # 추적 사용 횟수
self.detection_rejected = 0  # 탐지 거부 횟수
self.both_failed = 0  # 둘 다 실패 횟수
self.track_computed = 0  # 추적 계산 횟수
self.track_cached = 0  # 캐시 사용 횟수
# → 성능 분석을 위한 통계

# 라인 70-72: YOLO 워밍업
print("🔥 Warming up YOLO GPU...")
dummy = np.zeros((640, 640, 3), dtype=np.uint8)  # 가짜 이미지 생성
self.detector.predict(dummy, verbose=False)  # 한 번 실행
# → GPU 초기화 (첫 실행은 느림, 미리 실행)
```

🎯 init_tracker 메소드

목적: DaSiamRPN 추적기를 초기화 (첫 번째 박스로 시작)

```
def init_tracker(self, frame, bbox):
    """Initialize DaSiamRPN tracker"""
```

라인별 분석:

```
# 라인 78: bbox 분해
x, y, w, h = bbox
# → [100, 50, 30, 40] → x=100, y=50, w=30, h=40

# 라인 81-85: bbox 유효성 검사
h_frame, w_frame = frame.shape[:2]  # 프레임 크기
x = max(0, min(x, w_frame - 1))  # x를 0 ~ (너비-1) 범위로 제한
y = max(0, min(y, h_frame - 1))  # y를 0 ~ (높이-1) 범위로 제한
w = max(1, min(w, w_frame - x))  # w가 프레임 밖으로 안 나가게
h = max(1, min(h, h_frame - y))  # h가 프레임 밖으로 안 나가게
# → bbox가 이미지 범위 안에 있도록 보정

# 라인 87-88: 너무 작은 박스 거부
if w < 3 or h < 3:
    return False
# → 3x3픽셀보다 작으면 추적 불가

# 라인 92-95: bbox를 중심점 형식으로 변환
cx = x + w / 2.0  # 중심 x좌표
cy = y + h / 2.0  # 중심 y좌표
target_pos = np.array([cx, cy])  # 중심점
```

```
target_sz = np.array([w, h])  # 크기
# → DaSiamRPN은 중심점 방식을 사용

# 라인 98: DaSiamRPN 초기화
self.dasiamrpn_state = SiamRPN_init(frame, target_pos, target_sz, self.net)
# → 이 프레임에서 이 위치의 물체를 기억하라고 지시

# 라인 99-100: 상태 업데이트
self.tracking_initialized = True  # 추적 준비 완료
self.last_track_bbox = bbox  # 이 bbox 저장
# → 이제 추적 시작 가능
```

이 메소드가 하는 일:

1. 입력받은 bbox가 유효한지 확인
2. DaSiamRPN이 이해할 수 있는 형식으로 변환
3. DaSiamRPN에게 "이 물체를 추적해"라고 지시

🔄 track 메소드

목적: DaSiamRPN으로 현재 프레임에서 물체 위치 찾기

```
def track(self, frame):
    """Update DaSiamRPN tracker"""
```

라인별 분석:

```
# 라인 112-113: 추적 가능 여부 확인
if not self.tracking_initialized or self.dasiamrpn_state is None:
    return None
# → 추적이 초기화 안 됐으면 None 반환

# 라인 117: DaSiamRPN 실행
self.dasiamrpn_state = SiamRPN_track(self.dasiamrpn_state, frame)
# → "마지막 위치에서 이 프레임으로 이동했을 때 어디에 있나?"

# 라인 118: 통계 업데이트
self.track_computed += 1
# → 실제로 계산했다고 표시 (캐시 아님)

# 라인 121-122: 결과 추출
target_pos = self.dasiamrpn_state['target_pos']  # [cx, cy]
target_sz = self.dasiamrpn_state['target_sz']    # [w, h]
# → DaSiamRPN 결과에서 위치와 크기 가져오기

# 라인 125-129: 중심점을 박스로 변환
cx, cy = target_pos[0], target_pos[1]
w, h = target_sz[0], target_sz[1]
```

```
x = cx - w / 2.0  # 왼쪽 상단 x
y = cy - h / 2.0  # 왼쪽 상단 y
tracked_bbox = [int(x), int(y), int(w), int(h)]
# → [cx, cy, w, h] → [x, y, w, h]로 변환

# 라인 134-138: 결과 검증
h_frame, w_frame = frame.shape[:2]
if (x < -100 or y < -100 or w < 3 or h < 3 or
    x > w_frame + 100 or y > h_frame + 100):
    self.tracking_initialized = False
    return None
# → 결과가 말도 안 되면 (화면 밖으로 너무 나가ㅇ으면) 실패 처리

# 라인 140-141: 결과 저장 및 반환
self.last_track_bbox = tracked_bbox  # 다음에 사용할 수 있게 저장
return tracked_bbox
```

이 메소드가 하는 일:

1. DaSiamRPN을 실행하여 물체의 새 위치 계산
2. 결과를 bbox 형식으로 변환
3. 결과가 유효한지 확인
4. 다음에 사용할 수 있게 저장

🔍 detect 메소드

목적: YOLO로 현재 프레임에서 드론 탐지

```
def detect(self, frame):
    """Run YOLO detection"""
```

라인별 분석:

```
# 라인 149-153: YOLO 실행
results = self.detector.predict(
    frame,  # 입력 이미지
    conf=self.conf_threshold,  # 신뢰도 임계가ㅅ (0.5)
    verbose=False  # 출력 메시지 끄기
)
# → "이 이미지에서 드론을 찾아라" (신뢰도 0.5 이상만)

# 라인 155-156: 결과 확인
if len(results) == 0 or len(results[0].boxes) == 0:
    return None, 0.0
# → 아무것도 못 찾았으면 None, 0.0 반환

# 라인 158-160: 가장 좋은 탐지 선택
boxes = results[0].boxes  # 탐지된 모든 박스들
```

```
confs = boxes.conf.cpu().numpy()  # 신뢰도들
best_idx = int(np.argmax(confs))  # 가장 높은 신뢰도의 인덱스
# → 여러 개 탐지되면 가장 확실한 것 선택

# 라인 162-164: 선택된 박스 정보 추출
box = boxes[best_idx]  # 가장 좋은 박스
xyxy = box.xyxy[0].cpu().numpy()  # [x1, y1, x2, y2] 형식
conf = float(box.conf[0])  # 신뢰도
# → YOLO 결과 가져오기

# 라인 166-167: [x1,y1,x2,y2] → [x,y,w,h] 변환
x1, y1, x2, y2 = xyxy
bbox = [int(x1), int(y1), int(x2-x1), int(y2-y1)]
# → 우리가 사용하는 형식으로 변환

# 라인 169: 반환
return bbox, conf
# → bbox와 신뢰도 반환
```

이 메소드가 하는 일:

1. YOLO를 실행하여 드론 탐지
2. 여러 개 탐지되면 가장 확실한 것 선택
3. bbox 형식 통일
4. bbox와 신뢰도 반환

📏 calculate_distance 메소드

목적: 두 bbox 사이의 거리 계산 (중심점 간 거리)

```
def calculate_distance(self, bbox1, bbox2):
    """Calculate center distance between bboxes"""
```

라인별 분석:

```
# 라인 173-174: 유효성 검사
if bbox1 is None or bbox2 is None:
    return float('inf')
# → 둘 중 하나라도 None이면 무한대 반환 (멀리 떨어져 있다고 가ㄴ주)

# 라인 176-177: bbox 정보 추출
x1, y1, w1, h1 = bbox1
x2, y2, w2, h2 = bbox2
# → 두 박스의 좌표와 크기

# 라인 179-182: 중심점 계산
cx1 = x1 + w1 / 2.0  # bbox1의 중심 x
cy1 = y1 + h1 / 2.0  # bbox1의 중심 y
```

```
cx2 = x2 + w2 / 2.0  # bbox2의 중심 x
cy2 = y2 + h2 / 2.0  # bbox2의 중심 y
# → 가ㄱ 박스의 중심점 좌표

# 라인 184: 유클리드 거리 계산
return np.sqrt((cx1 - cx2)**2 + (cy1 - cy2)**2)
# → √((x차이)² + (y차이)²) = 두 점 사이 거리
```

예시:

```
bbox1 = [100, 100, 50, 50]  # 중심: (125, 125)
bbox2 = [200, 200, 50, 50]  # 중심: (225, 225)

거리 = √((225-125)² + (225-125)²)
    = √(100² + 100²)
    = √20000
    = 141.4 픽셀
```

🧠 process_frame 메소드 (가장 중요!)

목적: 매 프레임마다 실행되는 핵심 로직

이 메소드가 **전체 시스템의 두뇌**입니다!

```
def process_frame(self, frame):
```

전체 흐름도:

```
시작
 ↓
프레임 카운트 +1
 ↓
YOLO 탐지 실행 (매번)
 ↓
┌──────────────────────────────────┐
│ 추적 전략 결정                    │
├──────────────────────────────────┤
│ 추적 초기화 됐나?                 │
│  NO → track_bbox = None          │
│  YES → ▼                         │
│                                  │
│  탐지 실패했나?                   │
│   YES → 추적 실행 (필수!)         │
│   NO  → ▼                        │
│                                  │
│    주기 됐나? (3프레임마다)        │
```

```
│      YES → 추적 실행                │
│      NO  → 캐시 사용 (빠름!)       │
└──────────────────────────────────┘

 ↓
최종 bbox 결정
 ↓
┌──────────────────────────────────┐
│ Case 1: 둘 다 있음                 │
│  거리 체크                         │
│   > 100px → 추적 신뢰              │
│   ≤ 100px → 탐지 신뢰              │
├──────────────────────────────────┤
│ Case 2: 탐지만 있음                │
│  탐지 사용 + 추적 초기화           │
├──────────────────────────────────┤
│ Case 3: 추적만 있음                │
│  추적 사용                         │
├──────────────────────────────────┤
│ Case 4: 둘 다 없음                 │
│  LOST                              │
└──────────────────────────────────┘

 ↓
최종 bbox, 상태, 신뢰도 반환
```

라인별 상세 분석:

```python
# ============= 1단계: 준비 =============

# 라인 194: 프레임 카운터 증가
self.frame_count += 1
# → 몇 번째 프레임인지 기록

# ============= 2단계: YOLO 탐지 (항상) =============

# 라인 197: 탐지 실행
det_bbox, det_conf = self.detect(frame)
# → YOLO로 드론 찾기 (매 프레임)
# det_bbox: [x, y, w, h] 또는 None
# det_conf: 0.0 ~ 1.0 사이 신뢰도

# ============= 3단계: 추적 전략 결정 =============

# 라인 200: 추적 변수 초기화
track_bbox = None

# 라인 201: 추적이 초기화되어 있는지 확인
if self.tracking_initialized:
    # 추적 가능한 상태

    # 라인 202-205: 탐지 실패 시 전략
    if det_bbox is None:
        # 💡 중요: 탐지가 실패했으므로 추적에 의존해야 함
```

```
            track_bbox = self.track(frame)  # 추적 실행 (필수!)
            self.track_computed += 1  # 계산 횟수 +1

        # 라인 206-218: 탐지 성공 시 전략 (Sparse!)
        else:
            # 💡 중요: 탐지가 성공했으므로 추적은 가끔만 실행
            self.track_frame_count += 1  # 프레임 카운터 +1

            # 라인 210-214: 주기 체크
            if self.track_frame_count >= self.tracking_interval:
                # 💡 주기 도달 (예: 3프레임마다)
                track_bbox = self.track(frame)  # 추적 실행
                self.track_computed += 1
                self.track_frame_count = 0  # 카운터 리셋

            # 라인 215-218: 주기 아직 안 됨
            else:
                # 💡 캐시 사용! (속도 향상의 핵심)
                track_bbox = self.last_track_bbox  # 이전 결과 재사용
                self.track_cached += 1
                # → 계산 안 하고 저장된 가ㅅ 사용 (0ms!)

    # ============= 4단계: 최종 결정 =============

    # 라인 221-223: 결과 변수 초기화
    final_bbox = None  # 최종 bbox
    final_conf = 0.0   # 최종 신뢰도
    status = 'LOST'    # 기본 상태

    # ----- Case 1: 탐지와 추적 둘 다 성공 -----
    # 라인 225: 둘 다 있음
    if det_bbox is not None and track_bbox is not None:

        # 라인 227: 거리 계산
        distance = self.calculate_distance(det_bbox, track_bbox)
        # → 탐지 위치와 추적 위치가 얼마나 떨어져 있나?

        # 라인 229: 거리가 임계가ㅅ보다 큼 (점프 발생!)
        if distance > self.jump_threshold:
            # 💡 탐지가 가ㅂ자기 멀리 떨어진 곳을 가리킴
            # → 오탐일 가능성 높음
            # → 추적을 신뢰!

            final_bbox = track_bbox  # 추적 결과 사용
            status = 'TRACKED (det rejected)'  # 상태
            self.tracking_used += 1  # 통계
            self.detection_rejected += 1  # 거부 통계

            # 💡 여기서 추적 재초기화 안 함 (탐지가 잘못됨)

        # 라인 235: 거리가 가까움 (정상)
        else:
            # 💡 탐지와 추적이 비슷한 위치
            # → 탐지를 신뢰!
```

```
        final_bbox = det_bbox  # 탐지 결과 사용
        final_conf = det_conf  # 신뢰도
        status = 'DETECTED'  # 상태
        self.detection_used += 1  # 통계

        # 라인 243-244: 추적 재초기화
        self.init_tracker(frame, det_bbox)
        # → 좋은 탐지로 추적 업데이트
        self.track_frame_count = 0
        # → 카운터 리셋

# ----- Case 2: 탐지만 성공 -----
# 라인 246: 탐지만 있음
elif det_bbox is not None:
    # 💡 추적은 초기화 안 됐거나 실패
    # → 탐지 사용하고 추적 시작

    final_bbox = det_bbox
    final_conf = det_conf
    status = 'DETECTED'
    self.detection_used += 1

    # 라인 253-254: 추적 초기화
    self.init_tracker(frame, det_bbox)
    self.track_frame_count = 0

# ----- Case 3: 추적만 성공 -----
# 라인 256: 추적만 있음
elif track_bbox is not None:
    # 💡 탐지 실패, 추적만 성공
    # → 추적 결과 사용 (백업!)

    final_bbox = track_bbox
    status = 'TRACKED (no det)'
    self.tracking_used += 1

# ----- Case 4: 둘 다 실패 -----
# 라인 262: 둘 다 None
else:
    # 💡 완전히 놓침
    status = 'LOST'
    self.both_failed += 1

# ============= 5단계: 반환 =============

# 라인 266: 결과 반환
return final_bbox, status, final_conf
# → 이 프레임의 최종 결과
```

핵심 이해:

이 메소드는 **4가지 경우**를 다룹니다:

1. **둘 다 성공 + 가까움**: 탐지 사용 (가장 정확)
2. **둘 다 성공 + 멀리**: 추적 사용 (탐지 오류로 판단)
3. **탐지만 성공**: 탐지 사용
4. **추적만 성공**: 추적 사용 (백업)
5. **둘 다 실패**: LOST

실행 흐름도

🎬 프로그램 시작부터 종료까지

```
main() 함수
 ↓
명령줄 인자 파싱
 - --model: YOLO 모델 경로
 - --source: 비디오/카메라
 - --tracking-interval: 추적 주기
 - --jump-threshold: 점프 임계가ㅅ
 - --manual-init: 수동 초기화 여부
 ↓
SparseDaSiamRPNTracker 객체 생성
 - YOLO 로드
 - DaSiamRPN 로드
 - GPU 초기화
 ↓
run_inference() 함수 호출
 ↓
비디오/카메라 열기
 ↓
[선택] manual_init_select_bbox()
 - 사용자가 첫 bbox 선택
 - 추적 초기화
 ↓
메인 루프 시작
 │
 ├─▶ 프레임 읽기
 │
 ├─▶ process_frame(frame)
 │    ├─ YOLO 탐지
 │    ├─ 추적 전략 결정
 │    ├─ 결과 검증
 │    └─ 최종 bbox 결정
 │
 ├─▶ draw_bbox()
 │    - bbox 그리기
 │    - 상태 표시
 │    - 통계 표시
 │
 ├─▶ 화면 출력
 │
 ├─▶ [선택] 비디오 저장
 │
```

```
  ├─ ESC 키?
  │   YES → 종료
  │   NO  → 계속
  │
  └─▶ 반복
```

파라미터 조정 가이드

🎛️ tracking_interval (추적 주기)

의미: DaSiamRPN을 몇 프레임마다 실행할지

```
tracking_interval = 3  # 기본가ㅅ
```

조정 가이드:

값	속도	정확도	언제 사용?
2	중간 (45 FPS)	높음	빠른 움직임, 자주 가려짐
3	빠름 (50 FPS)	중간	**권장 (균형)**
5	매우 빠름 (60 FPS)	낮음	느린 움직임, 안정적 장면

코드 수정 위치:

```
# 라인 35
tracking_interval=3  # ← 여기를 수정

# 또는 실행 시:
python3 jetson_sparse_dasiamrpn_v3.py \
    --tracking-interval 5  # ← 명령줄에서 변경
```

🎯 jump_threshold (점프 임계값)

의미: 탐지가 얼마나 멀리 이동하면 "점프"로 간주할지 (픽셀)

```
jump_threshold = 100  # 기본가ㅅ
```

조정 가이드:

값	탐지 허용 범위	언제 사용?
50-100	엄격	느린 움직임, 오탐 많을 때

값	탐지 허용 범위	언제 사용?
100-200	중간	**권장 (일반)**
200-300	관대	빠른 움직임, 큰 가속도

판단 기준:

```
거리 < jump_threshold → 탐지 신뢰 (정상 이동)
거리 > jump_threshold → 추적 신뢰 (탐지 오류)
```

코드 수정 위치:

```
# 라인 33
jump_threshold=100  # ← 여기를 수정

# 또는 실행 시:
python3 jetson_sparse_dasiamrpn_v3.py \
    --jump-threshold 200  # ← 명령줄에서 변경
```

🎚️ conf_threshold (신뢰도 임계값)

의미: YOLO가 얼마나 확신할 때 탐지로 인정할지

```
conf_threshold = 0.5  # 기본가ㅅ (50%)
```

조정 가이드:

값	탐지 수	정확도	언제 사용?
0.3	많음	낮음 (오탐 많음)	작은 드론, 멀리 있을 때
0.5	중간	중간	**권장 (일반)**
0.7	적음	높음 (오탐 적음)	큰 드론, 가까이 있을 때

코드 수정 위치:

```
# 라인 33
conf_threshold=0.5  # ← 여기를 수정

# 또는 실행 시:
python3 jetson_sparse_dasiamrpn_v3.py \
    --conf 0.3  # ← 명령줄에서 변경
```

문제 해결 가이드

❓ 문제 1: 추적이 드론을 놓침

증상:

- 상태가 자주 "LOST"
- bbox가 드론을 따라가지 못함

원인:

1. `tracking_interval`이 너무 큼
2. 드론이 너무 빨리 움직임

해결:

```
# tracking_interval을 낮춤 (더 자주 추적)
python3 jetson_sparse_dasiamrpn_v3.py \
    --tracking-interval 2  # 3 → 2로 변경
```

설명: 더 자주 추적하면 드론의 빠른 움직임을 놓치지 않음

❓ 문제 2: FPS가 너무 낮음

증상:

- FPS가 30 이하
- 화면이 버벅임

원인:

1. `tracking_interval`이 너무 작음
2. GPU를 안 쓰고 있음

해결 1: Interval 증가

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --tracking-interval 5  # 3 → 5로 증가
```

해결 2: GPU 확인

```
# 라인 49-53 확인
if torch.cuda.is_available():  # GPU 있나?
    self.net = self.net.cuda()  # GPU 사용
    print("🚀 DaSiamRPN using CUDA")  # ← 이게 나와야 함
```

❓ 문제 3: 잘못된 물체를 탐지함

증상:

- bbox가 드론이 아닌 다른 물체로 이동
- "TRACKED (det rejected)" 자주 나옴

원인:

1. `jump_threshold`가 너무 높음
2. YOLO 신뢰도가 낮음

해결 1: Jump threshold 낮춤

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --jump-threshold 80  # 100 → 80으로 낮춤
```

해결 2: 신뢰도 높임

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --conf 0.7  # 0.5 → 0.7로 높임
```

❓ 문제 4: 작은 드론을 못 찾음

증상:

- 드론이 멀리 있을 때 탐지 안 됨
- 수동 초기화는 되는데 자동 탐지는 실패

원인:

- YOLO 신뢰도 임계값이 너무 높음

해결:

```
# 신뢰도 낮춤 (더 많이 탐지)
python3 jetson_sparse_dasiamrpn_v3.py \
    --conf 0.3  # 0.5 → 0.3으로 낮춤
    --manual-init  # 수동 초기화 사용
```

실전 사용 예시

📝 예시 1: 접근하는 드론 추적

상황: 드론이 멀리서 접근, 처음엔 작음

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --model best_fp16.engine \
    --source 0 \
    --manual-init \
    --conf 0.3 \
    --tracking-interval 3 \
    --jump-threshold 200
```

전략:

1. `--manual-init`: 작은 드론 수동 선택
2. `--conf 0.3`: 낮은 신뢰도도 허용 (작은 드론)
3. `--jump-threshold 200`: 관대하게 (빠른 접근)

📝 예시 2: 안정적인 장면 (최고 속도)

상황: 드론이 천천히 움직임, 가려짐 없음

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --model best_fp16.engine \
    --source 0 \
    --conf 0.5 \
    --tracking-interval 5 \
    --jump-threshold 150
```

전략:

1. `--tracking-interval 5`: 가끔 추적 (60 FPS)
2. 수동 초기화 불필요 (안정적 탐지)

📝 예시 3: 빠른 움직임 (최고 정확도)

상황: 드론이 빠르게 움직임, 급회전 많음

```
python3 jetson_sparse_dasiamrpn_v3.py \
    --model best_fp16.engine \
    --source 0 \
    --conf 0.5 \
    --tracking-interval 2 \
    --jump-threshold 300
```

전략:

1. `--tracking-interval 2`: 자주 추적 (정확도 우선)
2. `--jump-threshold 300`: 큰 이동 허용

통계 이해하기

프로그램 종료 시 나오는 통계:

```
📊 Performance Summary
============================================================
Total frames: 300

Detection: 255 (85.0%)        ← YOLO 사용
Tracking: 40 (13.3%)          ← DaSiamRPN 사용
Rejected: 5 (1.7%)            ← YOLO 거부
Lost: 0 (0.0%)                ← 둘 다 실패

DaSiamRPN computed: 105       ← 실제 계산
DaSiamRPN cached: 195         ← 캐시 사용
Cache efficiency: 65.0%       ← 캐시 효율

Average FPS: 52.3
Min FPS: 45.1
Max FPS: 58.7
============================================================
```

해석:

1. **Detection 85%**: 대부분 YOLO가 찾음 (좋음!)
2. **Tracking 13%**: 가끔 추적 사용 (정상)
3. **Rejected 2%**: 오탐 거의 없음 (좋음!)
4. **Cache 65%**: 캐시를 많이 사용 (빠름!)
5. **FPS 52**: 실시간 처리 가능 (좋음!)

최종 정리

🎯 핵심 개념 3가지

1. **Sparse Tracking**: 추적을 가끔만 실행하여 속도 향상
2. **Jump Detection**: 갑작스런 이동 감지하여 오탐 방지
3. **Fallback Strategy**: 탐지 실패 시 추적으로 백업

📌 중요 파라미터 3가지

1. `tracking_interval`: 속도 vs 정확도 (권장: 3)
2. `jump_threshold`: 오탐 방지 민감도 (권장: 100-200)
3. `conf_threshold`: 탐지 신뢰도 (권장: 0.5)

🔧 수정할 때 주의사항

1. **한 번에 하나씩 변경:** 어떤 파라미터가 영향을 주는지 확인
2. **테스트 후 조정:** 실제 영상으로 테스트하고 미세 조정
3. **통계 확인:** 프로그램 종료 시 통계로 성능 파악

이 매뉴얼로 코드의 모든 부분을 이해하고 필요에 맞게 조정할 수 있습니다! 🚁✨